ChatGPT

# Comprehensive Audit of the Design Space Project

## Introduction

This audit reviews the current state of the **Design Space** repository (**maplebakin/designspace**) as of **January 23 2026**. The project is a browser-based design studio for print and digital layouts using React 18, TypeScript, Vite, Fabric.js, Tailwind CSS, Zustand and several supporting libraries. Features include a drag-and-drop canvas editor, layers panel, smart guides, templates, sticker import, a **Brand Vault** for themes and exports to PNG/JPG/SVG/PDF [1]. Recent internal documents (REFACTOR_DS.md, SERVICE_EXTRACTION_SUMMARY.md, CANVAS_SERVICE_REFACTOR.md) outline a **forensic audit** and a roadmap to stabilise the canvas state and extract business logic from UI components. The following assessment summarises current implementation status, identifies barriers, and proposes upgrades and innovative enhancements.

## Current Architecture and Implementation

### Codebase Structure

The repository uses a **React 18** frontend with **Vite** and **TypeScript**. **Tailwind CSS 3.4** provides styling with custom theme variables defined in `tailwind.config.js`, and the code emphasises composability through components and hooks. **Zustand** manages application state, while **Fabric.js** renders and manipulates canvas objects. **pdf-lib** and **pdfjs-dist** enable PDF import/export. The **Brand Vault** persists theme tokens, and the editor supports image/PDF/SVG imports via an **asset loader** service. Project configuration files (`tsconfig.json`, `tailwind.config.js`, etc.) enforce strict type checking and custom UI themes.

### Forensic Audit Findings

The **Heart-Beat Restoration Roadmap** (dated **Jan 9 2026**) describes critical problems discovered during a forensic audit:

1. **Dual sources of truth** – The Fabric canvas and the Zustand store can diverge. Race conditions during initialization or theme application cause inconsistent state and infinite update loops [2]. The roadmap advocates making Zustand the **single source of truth** and letting Fabric act solely as a rendering delegate.
2. **Race conditions and memory leaks** – The current initialization sequence allows asynchronous handlers to run against an uninitialized canvas, leading to memory leaks and event handlers that persist after unmount. Theme application can also run before the canvas is ready [2].
3. **Infinite re-render loops** – Bidirectional updates between Fabric and Zustand cause loops. Smart-guide filtering and RAF scheduling were scattered across components, exacerbating performance problems.

4. **Coordinate & layering inconsistencies** – Fabric uses mixed units (pixels, percentage, viewport units) and lacks a unified coordinate system. Guides sometimes overlap layers or render at the wrong scale. Canvas offset calculations drift over time and scale zero values are not guarded.

### Refactor & Service Extraction Work

The roadmap proposes four phases:

1. **Initialization & Disposal Safety** – create an atomic initialization sequence, cancellation tokens, and gating for theme application. The code has begun to implement this. Functions such as `applyActiveThemeToCanvas` now check `canvasReadyState` before applying a theme and acquire a sync lock [3].
2. **Source of Truth Consolidation** – introduce a serialized `canvasObjects` store in Zustand with dual-write mode and a sync-lock to avoid interference. Event handlers are being moved out of components. The `GuideRegistry` class centralizes guide registration and filtering [4].
3. **Stabilization of the Update Loop** – unify RAF scheduling and introduce a circuit breaker to halt infinite loops. The `GuideRegistry` and the `canvasEventService` are early steps in this direction.
4. **Coordinate & Layering Fixes** – design a `CoordinateSystem` class, define explicit z-index manifests, improve canvas offset calculation using `ResizeObserver`, and guard against zero-scale values.

Additional internal refactors demonstrate progress:

- **Service extraction** – Business logic for theme validation and export has been moved into `themeValidationService.ts` and `exportService.ts`. This reduced code in UI components by 49 % and 40 %, improved testability, and decoupled logic from presentation [5] [6].
- **Canvas Event Service** – A new `canvasEventService.ts` centralizes event registration and cleanup. It registers handlers with abort signals and provides a single `cleanupAll()` call to remove them [7]. This eliminates manual cleanup lines and prevents memory leaks [8].
- **Guide Registry** – `guideRegistry.ts` introduces a registry to track guide objects, support backward compatibility for legacy `isGuide` flags, and filter objects by type [4]. This centralization is part of the Phase 3 stabilization.

These refactors show the team is actively addressing the roadmap's pain points. However, many tasks remain incomplete, and some barriers persist.

## Identified Barriers & Pain Points

1. **Synchronization and single source of truth** – The canvas still holds its own state, and the dual-source architecture remains in progress. Without fully consolidating objects into Zustand, race conditions may reappear. Implementation of dual-write mode, sync locks, and a `canvasObjects` store needs completion.
2. **Infinite update loops** – While `GuideRegistry` and `canvasEventService` reduce loops, there is no unified RAF scheduler or circuit breaker yet. Without those, new features or third-party code could reintroduce loops.

3. **Coordinate drift and layering** – The project currently uses Fabric's default coordinate system, which mixes units and leads to offset drift. The `CoordinateSystem` class, z-index manifest and canvas offset recovery logic have not yet been implemented.
4. **Theme and Asset Consistency** – The asset loader handles images, SVGs and PDFs, but heavy assets can slow down the editor. Themes rely on user-provided tokens; there is no fallback for invalid tokens, and dark-mode accessible themes are not enforced.
5. **Outdated dependencies** – Many dependencies are pinned to versions from 2023/2024. Upgrading can unlock performance improvements and new features:
6. **React 18 → 19.2** – React 19.2 (released Oct 1 2025) introduces `<Activity/>` for pre-rendering hidden UI and deferring updates, the `useEffectEvent` hook to avoid re-running effects unnecessarily, `cacheSignal` for React Server Components, partial pre-rendering, and a built-in **compiler** that eliminates the need for `useMemo` / `useCallback` by automatically optimizing components [9] . The new **Actions API** simplifies form handling and automatically tracks loading/error states [10] .
7. **Tailwind CSS 3.4 → 4.0** – Tailwind 4.0 (Jan 22 2025) is a ground-up rewrite with a high-performance engine; full builds are up to **5× faster** and incremental builds over **100× faster** [11] [12] . It introduces modern CSS features like cascade layers, custom properties, `color-mix()`, container queries, 3D transforms, and dynamic utility values [13] . Installation is greatly simplified, requiring only a single `@import` in CSS and no configuration [14] . A first-party Vite plugin further improves performance [15] .
8. **Zustand 4 → 5** – Zustand v5 (Oct 20 2024) does not add new features but removes deprecated ones and drops support for React < 18. It eliminates default exports, requires React 18 and TypeScript 4.5, and uses native `useSyncExternalStore`, reducing bundle size [16] . Migration is straightforward but requires replacing certain APIs and using the `useShallow` hook to avoid infinite loops.
9. **Fabric.js 6 Beta → 6.x stable / 7.0** – The project uses `fabric` 6.0.0-beta18. Fabric.js 6 is now stable and fully modular; it is rewritten in TypeScript, uses named ES6 imports, replaces `fabric.util.createClass` with native classes, returns promises instead of callbacks, and discourages method chaining [17] . Upgrading eliminates the global `fabric` object and offers built-in TypeScript types [17] . Fabric 7.0 (preview) mainly raises the minimum Node version to 20 and changes default `originX` / `originY` values to `'center'` [18] , so it should not block migration.
10. **Other libraries** – Upgrading `pdf-lib`, `pdfjs-dist`, Dexie/idb, and lodash can bring security and performance improvements. Some libraries may also have modern tree-shakeable builds.
11. **Testing and observability** – The roadmap calls for unit, integration, visual regression, and performance tests, but the repository currently lacks a comprehensive test suite. Without automated tests, regressions or infinite loops could slip into production. Metrics (e.g., frame rate, memory usage) are not collected, hindering performance tuning.
12. **Cross-browser and accessibility issues** – Fabric.js upgrade notes highlight changes in default object origins and event behaviour. Without careful migration, objects may render off-screen or events may fire unexpectedly. Additionally, accessible design (keyboard navigation, ARIA roles) is not emphasized, limiting use by users with disabilities.

# Implementation Recommendations

## Complete the Single Source of Truth Migration

- **Finish Phase 2 tasks** – Implement the `canvasObjects` store in Zustand as the authoritative list of objects. Use a **dual-write** mode where operations update both the store and the canvas until the

migration is complete. Add a **sync-lock** counter to prevent concurrent writes (already used in `applyActiveThemeToCanvas` [3] ). Once stable, switch Fabric into a pure rendering delegate that reads from Zustand.

- **Adopt the unified RAF scheduler and circuit breaker** – Build a `FrameScheduler` to queue canvas updates and throttle them. Integrate a circuit breaker with exponential backoff to halt infinite loops (Phase 3). Expose metrics (frame duration, dropped frames) to help tune performance.
- **Implement CoordinateSystem and z-index manifest** – Introduce a `CoordinateSystem` class that normalises units, scales and offsets. Define explicit z-index values for guides, selection handles, shadows and layers to prevent overlap and drift. Use `ResizeObserver` to track canvas size changes and update offsets.

## Modernise the Technology Stack

- **Upgrade to React 19.2** – This yields performance improvements through the built-in compiler (no more manual `useMemo` / `useCallback` ) and new primitives. `<Activity/>` allows pre-rendering hidden tabs so that navigation between layers or panels is instant [9] . The **Actions API** simplifies forms and reduces boilerplate; Design Space's export/import modals and theme editors could benefit from this API [10] . The `useEffectEvent` hook prevents unnecessary re-runs of effects when dependencies change [19] .
- **Adopt Tailwind CSS 4** – Upgrading provides a faster build pipeline with up to **5× faster** full builds and more than **100× faster** incremental builds [11] [12] . The new engine uses Lightning CSS and eliminates configuration overhead; combined with the first-party Vite plugin, it will significantly shorten local development time [14] [15] . New features like container queries, `color-mix()` , cascade layers, dynamic utilities and 3D transforms enable advanced responsive and interactive layouts [13] . These could be used to provide contextual toolbars or responsive side panels.
- **Migrate to Zustand 5** – The project already uses React 18; migrating to Zustand 5 removes legacy exports, reduces bundle size and uses the native `useSyncExternalStore` API. The migration guide provides patterns for replacing deprecated features and avoiding infinite loops via `useShallow` .
- **Upgrade to stable Fabric.js** – Moving from 6.0.0-beta18 to the latest 6.x stable (or even 7.0) eliminates beta bugs and leverages modern TypeScript builds. The upgrade requires replacing `fabric` global imports with named imports, adjusting code for ES6 classes, awaiting asynchronous methods, and avoiding method chaining [17] . For Fabric 7, update Node to 20 and be aware that objects' default `originX` / `originY` have switched to `'center'` [18] ; use `getPositionByOrigin` and `positionByLeftTop` helpers to convert positions as needed. Also handle changed event defaults for middle/right click and gradient opacity [20] .
- **Refresh other dependencies** – Upgrade `pdf-lib` and `pdfjs-dist` to their latest versions to benefit from security patches and performance optimizations. Modern `pdf-lib` releases support incremental updates and better font embedding; `pdfjs-dist` continues to improve text selection and performance. Evaluate replacing `idb` /Dexie with built-in IndexedDB wrappers if they become available in browsers.

## Improve Testing, Observability and Code Quality

- **Automated test suite** – Establish unit tests for services (theme validation, export, asset loading), integration tests for editing flows (adding objects, undo/redo, exports) and visual regression tests (e.g., using Playwright and storybook). Add performance tests to measure frame rate and memory usage before and after major changes.

- **Telemetry and instrumentation** – Collect metrics such as canvas render time, commit duration, memory usage and network bandwidth. Use these to detect regressions and to validate the benefits of upgrades (e.g., React 19 or Tailwind 4). Consider using `performance.mark()` and `performance.measure()` along with logs.
- **Accessibility** – Audit the UI for keyboard navigation, ARIA roles and colour contrast. Provide focus rings on interactive elements, support screen readers, and respect prefers-reduced-motion settings. This makes the editor inclusive and can expand the user base.
- **Continuous integration** – Use GitHub Actions to run linting, type checking, tests and build on each pull request. Introduce commit hooks (Husky) to enforce code formatting and commit message conventions.

### Innovate Beyond the Roadmap

While the roadmap focuses on stabilising the core, several **out-of-the-box** enhancements could make Design Space more competitive and delightful:

- **AI-assisted layouts and design suggestions** – Integrate a generative model or rule-based engine that suggests layouts, colour palettes, or typographic pairings based on the user's current design. For example, given a set of objects on the canvas, suggest balanced alignments or complementary colours from the Brand Vault.
- **Collaborative editing** – Implement real-time collaboration using CRDTs (Conflict-free Replicated Data Types) or Operational Transformation. This would allow multiple users to co-edit a canvas, making Design Space suitable for remote teams.
- **Undo/Redo history snapshots** – Provide visual thumbnails for each history state. A user could hover over a snapshot to preview the canvas before committing to an undo/redo action.
- **PWA & offline support** – Turn Design Space into a Progressive Web App so that users can install it and continue working offline. Leverage service workers to cache assets and store drafts in IndexedDB; sync changes when back online.
- **Template & asset marketplace** – Allow users to browse and import community templates, fonts, and stickers directly within the app. Coupled with AI search, this could accelerate design workflows.
- **Accessibility modes & theming** – Offer high-contrast and dyslexia-friendly themes. Provide an **auto-dark/light** theme that responds to OS preferences. Use Tailwind 4's colour-mix and custom properties to generate accessible colour schemes [13].
- **Advanced exports** – Support multi-page documents, bleed and margin settings, and integration with print providers. Provide an export preview with printer marks and colour profiles.
- **Plugin architecture** – Expose APIs so that third-party developers can write plugins (e.g., to import PSD files, add shape libraries or connect to DAM systems). A plugin sandbox with permissions would maintain security.

## Conclusion

Design Space is an ambitious canvas editor with a powerful technology stack. Internal audits have identified critical architectural issues around state synchronization and event handling, and the team has begun refactoring to address them. Completing the roadmap, upgrading dependencies (React 19, Tailwind 4, Zustand 5, Fabric 6/7), enhancing testing and observability, and embracing innovative features like AI-assisted layouts and collaboration will strengthen the product and position it for future success. Each recommended upgrade offers tangible benefits — faster builds, cleaner code, improved performance and

new capabilities — that outweigh the migration effort. By prioritising stability while exploring creative improvements, Design Space can continue to evolve into a robust, delightful tool for designers and non-designers alike.

---


1 README.md
https://github.com/maplebakin/designspace/blob/main/README.md

2 REFACTOR_DS.md
https://github.com/maplebakin/designspace/blob/main/REFACTOR_DS.md

3 themeUtils.ts
https://github.com/maplebakin/designspace/blob/main/src/editor/fabric/themeUtils.ts

4 guideRegistry.ts
https://github.com/maplebakin/designspace/blob/main/src/editor/fabric/guideRegistry.ts

5 6 SERVICE_EXTRACTION_SUMMARY.md
https://github.com/maplebakin/designspace/blob/main/SERVICE_EXTRACTION_SUMMARY.md

7 8 CANVAS_SERVICE_REFACTOR.md
https://github.com/maplebakin/designspace/blob/main/CANVAS_SERVICE_REFACTOR.md

9 10 React 19.2 is Stable! What's Actually New in 2025 | by Alexander Burgos | Medium
https://medium.com/@alexdev82/react-19-2-is-stable-whats-actually-new-in-2025-1e2056cb74b1

11 12 13 14 15 Tailwind CSS v4.0 - Tailwind CSS
https://tailwindcss.com/blog/tailwindcss-v4

16 Announcing Zustand v5 | Poimandres
https://pmnd.rs/blog/announcing-zustand-v5

17 Starting with Fabric.js v6 — What's New and How to Use It Without Getting Confused | by The Coder | Medium
https://medium.com/@samruddhi.thakre0111/starting-with-fabric-js-v6-whats-new-and-how-to-use-it-without-getting-confused-2f14a370ba7d

18 20 Upgrading to Fabric.js 7.0 | Docs and Guides
https://fabricjs.com/docs/upgrading/upgrading-to-fabric-70/

19 React 19.2 – React
https://react.dev/blog/2025/10/01/react-19-2